Rich Text / Markdown Editor with Preview

- **Problem:** Build a Markdown editor with a live preview. The user types Markdown on the left and sees formatted output on the right. A toolbar (and keyboard shortcuts) applies bold, italic, strikethrough, inline code, links, headings, quotes and lists to the selected text. The draft survives a page refresh. The preview must never run scripts the user (or a pasted document) puts in the text.
- **Tech stack:** React (Vite, JavaScript, Tailwind CSS v4)
- **Time limit:** 60 minutes
- **Often asked at:** Atlassian, Notion, Microsoft
- **Why it matters:** contentEditable, safe HTML (XSS)

PART A: Thought process

General steps for any machine coding task

1. **Understand the requirements.** Restate the problem, split functional vs non-functional, ask clarifying questions, write down assumptions.
2. **List the interactions.** Every user action and its expected result. This is your checklist for building and testing.
3. **Design the architecture.** Name the components and their one-line jobs. Start small.
4. **Design the state.** What is the single source of truth? What can be derived?
5. **Design data structures and contracts.** Shape of the data, function inputs and outputs.
6. **Trace one data flow end to end.** Proves the design works before you type.
7. **Find the hard parts.** Solve them on paper first.
8. **Build in layers.** Simplest working version first, then one working layer at a time.
9. **Cover states and edge cases.** Empty, error, huge input, weird selections, malicious input.
10. **Wrap up.** Trade-offs and what you would add with more time.

Time budget for 60 minutes

Phase	Time
Requirements + clarifying questions (textarea vs contentEditable decided here)	5 min
Interactions, architecture, state design	5 min
Hard parts on paper (XSS, selection edits + undo, parser order)	5 min
Build: textarea + preview with a tiny parser (headings, paragraphs, bold)	10 min
Build: escaping, safe links, inline code, lists, quotes, code blocks	12 min
Build: toolbar + shortcuts with pure edit functions	12 min
Build: autosave, view modes, list continuation, word count	6 min
Edge cases, accessibility, wrap-up	5 min

Tip: think out loud

Say what you are deciding and why: "I will keep the Markdown string as the only source of truth and treat the preview as a pure function of it. The text goes into a textarea, not a contentEditable div, so the browser never invents HTML for me. And every character of user text is escaped before I add any tag, so the preview can't run a script even if someone types one." The interviewer grades your reasoning as much as the code.

PART B: Apply it to this problem

1. Understand the requirements

Functional requirements

- A text area to write Markdown.
- A live preview that updates as the user types.
- Supported syntax: headings `#` .. `######`, paragraphs, **bold**, *italic*, ~~strikethrough~~, `inline code`, links, bulleted / numbered / task lists, blockquotes, fenced code blocks, horizontal rules.
- A toolbar: Bold, Italic, Strikethrough, Code, Link, Heading, Quote, Bulleted list, Numbered list, Task list.
- Toolbar buttons act on the **selected text**: wrap it, or prefix the selected lines. Pressing the same button again removes the formatting.
- Keyboard shortcuts: Ctrl/Cmd + B, I, K.
- Pressing Enter inside a list item starts the next item. Enter on an empty item leaves the list.
- Three views: Write, Split, Preview.
- The draft autosaves and is restored after a refresh. Show "Saving… / Saved".
- Word and character count.

Non-functional requirements

- **Security:** no XSS. `<script>`, `<img onerror>`, `javascript:` links must show as text or become harmless.
- **Performance:** typing never lags, even on a long document. The preview may be a few milliseconds behind.
- **Correctness of editing:** Ctrl+Z undoes toolbar actions too, not only typed characters. The selection lands where the user expects after every action.
- **Error handling:** storage full or blocked must not crash the editor; the user sees "Not saved".
- **Accessibility:** a labelled textarea, a `role="toolbar"` with labelled buttons, shortcuts shown in tooltips, `aria-pressed` on the view switch, save status in a live region. IME input (Chinese, Japanese) must not break.
- **Reusability / extensibility:** the parser and the edit functions are pure functions with no React in them. Adding a toolbar button is adding one entry to a `TOOLS` array.

Clarifying questions to ask the interviewer

1. Is it a **Markdown** editor with a preview, or a **WYSIWYG** rich text editor (like Google Docs) where you format text in place? This decides textarea vs contentEditable.
2. Which Markdown features are in scope? Tables, images, nested lists, HTML blocks?
3. Can I use libraries like `marked` and `DOMPurify`, or should I write the parser myself?

4. Should single line breaks show as line breaks (like GitHub comments) or be joined (like strict CommonMark)?
5. Where is the document saved? Only locally, or to a server? Is there collaboration?
6. Who sees the rendered HTML? Only the author, or other users too (which makes XSS much more serious)?

Assumptions we will build with

- It is a **Markdown editor with a split preview**, not WYSIWYG. Source is a `<textarea>`. (Section 7 explains why, and what changes for WYSIWYG.)
- **No libraries**: I write a small parser that covers the list above. I will say that in production I'd use `marked` + `DOMPurify`.
- Single line breaks inside a paragraph become `<br>` (GitHub comment style). It is what users of an editor expect.
- One document, saved in `localStorage`. No server, no collaboration.
- Raw HTML in the Markdown is **not** allowed: it is shown as text.
- Links open in a new tab with `rel="noopener noreferrer"`. Only `http`, `https`, `mailto` and relative URLs are allowed.

---

2. Identify interactions / use cases

Action / input	Expected behavior / output
Type in the textarea	Text updates instantly; preview follows; status shows "Saving…" then "Saved"
Type `# Title`	Preview shows an `<h1>`
Type `<img src=x onerror=alert(1)>`	Preview shows that text literally; no alert
Type `[x](javascript:alert(1))`	Preview shows a link with `href="#"`; clicking does nothing
Select "hello", click **B**	Text becomes `**hello**`; "hello" stays selected
Click **B** again with "hello" still selected	Markers are removed; back to `hello`
Click **B** with nothing selected	Inserts `**bold text**` with "bold text" selected, ready to type over
Double-click a word (selects trailing space), click **B**	`**word**` not `**word **`
Select 3 lines, click "Bulleted list"	Each line gets `- `; click again removes them
Click "Numbered list" on 3 lines	`1. `, `2. `, `3. `
Select "docs", click Link (or Ctrl+K)	`[docs](https://)` with `https://` selected
Ctrl+Z after a toolbar action	The toolbar action is undone, like any typing
Press Enter at the end of `- item`	New line `- `
Press Enter at the end of `3. item`	New line `4. `

Action / input	Expected behavior / output
Press Enter on an empty `-`	Marker is removed, list ends
Press Enter while typing Japanese (IME composing)	Normal IME behavior, no list logic
Click Write / Split / Preview	Only the editor / both / only the preview
Refresh the page	Last draft is restored
Close the tab right after typing	The `pagehide` handler saves immediately
Storage full or blocked	Status says "Not saved: storage is full or blocked"; editing still works
Clear all text	Preview says "Nothing to preview yet"; count shows 0 words

This table is the build checklist and the test list.

---

3. Design the architecture

Module	Responsibility
`App`	Owns the document text and the view mode. Lays out editor, preview, footer.
`Editor`	The textarea. Runs toolbar tools and shortcuts, applies edits to the selection, continues lists on Enter.
`Toolbar`	Renders one button per tool from the `TOOLS` config. Knows nothing about text.
`Preview`	Turns Markdown into HTML (memoized) and renders it.
`utils/markdown.js`	Pure parser: `markdownToHtml`, `inline`, `escapeHtml`, `safeUrl`. All XSS rules live here.
`utils/editing.js`	Pure edit functions: `toggleWrap`, `toggleLinePrefix`, `insertLink`, `continueList`, and the `TOOLS` list.
`hooks/useDraft.js`	Text state + debounced autosave to `localStorage` + save on `pagehide`.

```
App  (text via useDraft, view, deferredText)
├─ Editor  (value, onChange)
│  ├─ Toolbar  (onTool)  ── reads TOOLS
│  └─ <textarea>  ── edits come from utils/editing.js
├─ Preview  (source)  ── utils/markdown.js
└─ footer  (word count, save status)
```

Why this split: the two hard pieces of logic (parsing, editing a selection) are pure functions, so I can test them with plain asserts and no browser. React components stay thin. I started with everything in `App` and split out `Editor` only when the toolbar needed the textarea ref.

---

4. Design the state / data model

State or field	Purpose
`text` (string)	The Markdown source. **The only source of truth.**
`saveStatus` (`'saved' | 'saving' | 'error'`)	What the footer shows
`view` (`'write' | 'split' | 'preview'`)	Which panes are visible
`deferredText`	A lagging copy of `text` from `useDeferredValue`, used by the preview
`ref` to the textarea	Needed to read and set the selection; not React state

```
const [text, setText, saveStatus] = useDraft('md-editor:draft', SAMPLE);
const [view, setView] = useState('split');
const deferredText = useDeferredValue(text);
```

Senior note: derived vs stored, and who owns what

- **HTML is derived, never stored.** `html = markdownToHtml(text)`. Storing it would mean two copies that can drift apart. It is recomputed in `Preview` with `useMemo`.
- **Word count and character count are derived** in render.
- **The selection is not React state.** The textarea already owns it (`selectionStart`, `selectionEnd`). Copying it into state on every `select` event would re-render on every arrow key and could fight the browser. We read it only at the moment a tool runs.
- **Undo history is not our state.** The browser keeps it for the textarea, as long as we edit through `execCommand('insertText')` (Challenge 3).
- `App` owns `text` because both `Editor` and `Preview` need it. `useDraft` owns persistence so `App` doesn't know about `localStorage`.

---

5. Design the data structures and contracts

The document

Just a string. No AST stored. Example:

```
## Checklist
- [x] Toolbar with shortcuts
- [ ] Press Enter in a list to continue it

Read the [Markdown guide](https://www.markdownguide.org).
```

Parser contract

```
markdownToHtml(source: string): string   // safe HTML for the preview
inline(line: string): string             // one line of inline syntax, escaped
escapeHtml(s: string): string            // & < > " ' to entities
safeUrl(url: string): string             // url, or "#" if the scheme is not allowed
```

Example:

```
markdownToHtml('## Hi **you**\n<b>x</b>')
// '<h2>Hi <strong>you</strong></h2>\n<p>&lt;b&gt;x&lt;/b&gt;</p>'
```

Edit contract (the key design choice)

Every toolbar action is a pure function `(value, selectionStart, selectionEnd) => Edit`:

```
// Edit: "replace value[from, to) with insert, then select [selStart, selEnd)"
{ from: 4, to: 9, insert: '**hello**', selStart: 6, selEnd: 11 }
```

Why a range and not the whole new string? Because a range can be applied with `document.execCommand('insertText')`, which keeps Ctrl+Z working. A whole new string can only be applied with `setValue`, which wipes the browser's undo history.

Tool config

```
{ id: 'bold', label: 'Bold', icon: 'B', key: 'b', run: (v, s, e) => toggleWrap(v, s, e, '**', 'bold
text') }
```

`label` is the accessible name, `key` is the Ctrl/Cmd shortcut, `run` returns an Edit. The toolbar and the shortcut handler read the same list, so they can't disagree.

Storage contract

`localStorage['md-editor:draft'] = '<markdown string>'`. The key is namespaced so other apps on the same origin don't clash. No server API.

---

6. Trace the data flow

The user selects "hello" in `say hello now` and presses Ctrl+B.

```
keydown Ctrl+B on <textarea>
  ↓ Editor.onKeyDown finds TOOLS entry with key 'b', preventDefault()
  ↓ read selection from the DOM: selectionStart = 4, selectionEnd = 9
  ↓ toggleWrap('say hello now', 4, 9, '**')            (pure)
  ↓ returns { from: 4, to: 9, insert: '**hello**', selStart: 6, selEnd: 11 }
  ↓ apply(edit): setSelectionRange(4, 9) → execCommand('insertText', '**hello**')
```

```
  ↓ browser replaces the text, records an undo step, fires an 'input' event
  ↓ React onChange → useDraft.update('say **hello** now') → status 'saving'
  ↓ apply() sets the selection to [6, 11]  ("hello" selected again)
  ↓ re-render: textarea already shows the value; Preview gets deferredText
  ↓ when React is idle: useMemo → markdownToHtml → '<p>say <strong>hello</strong> now</p>'
  ↓ 500 ms after the last change: localStorage.setItem(...) → status 'saved'
output: bold "hello" in the preview, "Saved" in the footer, Ctrl+Z restores 'say hello now'
```

---

7. Important technical challenges

Challenge 1: Rendering user content without XSS

Problem. The preview has to become real HTML, so at some point we use `dangerouslySetInnerHTML`. If the user types `<img src=x onerror="alert(document.cookie)">`, or pastes a document that contains it, a naive parser passes it through and the browser runs it. Links are a second hole: `[click](javascript:alert(1))` produces a working `javascript:` URL. If the document is ever shown to another user (shared notes, comments), this is a stored XSS.

Solution.

1. **Escape first, then add tags.** Every piece of user text goes through `escapeHtml` *before* any regex adds `<strong>`, `<a>`, etc. So the only tags in the output are the ones our code writes. There are exactly two places text enters the output: `inline()` and the code block body, and both escape.
2. **Allow-list URL schemes.** Allow `http`, `https`, `mailto` and relative URLs. Everything else (`javascript:`, `data:`, `vbscript:`) becomes `#`. Strip control characters first, because browsers ignore them inside a scheme (`java\u0001script:` still runs).
3. **Attribute values can't break out.** `"` is escaped to `"`, and the link regex doesn't allow spaces in the URL, so `" onmouseover="...` can't become an attribute. The code-block language is limited to `[\w-]`.
4. `target="_blank"` links get `rel="noopener noreferrer"` so the new page can't control ours through `window.opener`.
5. In production: use `marked` (or `markdown-it`) **and** run the output through `DOMPurify`, plus a Content-Security-Policy that blocks inline scripts. Defence in depth.

```
const ESCAPES = { '&': '&amp;', '<': '&lt;', '>': '&gt;', '"': '&quot;', "'": '&#39;' };
export const escapeHtml = (s) => s.replace(/[&<>"']/g, (c) => ESCAPES[c]);

export function safeUrl(url) {
  const clean = url.replace(/[\u0000- \u007F]/g, '');
  const scheme = clean.match(/^([a-z][a-z0-9+.-]*):/i);
  if (!scheme) return url;                      // relative link
  return /^(https?|mailto)$/i.test(scheme[1]) ? url : '#';
}
```

Note the order matters: we escape, *then* check the URL. `javascript:` can't sneak through, because its `&` is already `&`, so the browser sees the literal text `javascript:`, which is a harmless relative URL.

Challenge 2: textarea + preview, or contentEditable?

Problem. The obvious idea for "rich text" is `<div contentEditable>` and `document.execCommand('bold')`. In practice:

- Each browser produces different HTML for the same action (`<b>` vs `<strong>` vs `<span style="font-weight:bold">`, `<div>` vs `<p>` vs `<br>` on Enter).
- Pasting from Word or a web page brings in arbitrary HTML, styles, and possibly `onerror` handlers. That HTML is now *your document*.
- React and contentEditable fight: if React re-renders the div's children, the caret jumps to the start. You end up making it uncontrolled and syncing by hand.
- The DOM becomes the source of truth, so saving, undo, collaboration and validation all have to parse messy HTML.

Solution. For a Markdown editor, use a plain `<textarea>`. The source is a string, the browser handles caret, selection, IME, spellcheck, undo and accessibility for free, and paste is plain text by definition. The preview is a pure function of the string.

If the interviewer wants **WYSIWYG**, the senior answer is: keep a **document model** (a tree of blocks and text runs with marks) as the source of truth, render it into contentEditable, intercept `beforeinput` events and turn them into model operations, and map DOM selection to model positions. That is what ProseMirror, Lexical, Slate and Notion do. In 60 minutes I'd use one of them. If I must hand-roll contentEditable, at minimum make paste plain text:

```
<div
  contentEditable
  suppressContentEditableWarning
  onPaste={(e) => {
    e.preventDefault();                                  // never accept pasted HTML
    const text = e.clipboardData.getData('text/plain');
    document.execCommand('insertText', false, text);
  }}
/>
```

Challenge 3: Toolbar edits that keep the selection and Ctrl+Z

Problem. The simple version is `setText(before + '**' + selected + '**' + after)`. Three things break:

1. **Undo is lost.** Setting a textarea's value from JavaScript clears the browser's undo history. The user presses Ctrl+Z after clicking Bold and nothing happens, or worse, all their typing is gone.
2. **Selection is lost.** After React sets the value, the caret jumps to the end.
3. **Clicking the button blurs the textarea**, and toggling (pressing Bold twice) needs to recognise markers that are *outside* the selection (`**[hello]**`) as well as *inside* it (`[**hello**]`).

Solution.

- Pure `toggleWrap` / `toggleLinePrefix` return an Edit range (section 5). Easy to unit-test, no DOM.
- `apply` selects the range and calls `execCommand('insertText')`. The browser records it as one undo step and fires a normal `input` event, so React's `onChange` runs as if the user typed it. Then set the new

selection. Fall back to `setRangeText` where `execCommand` returns `false`.
- `onMouseDown={e => e.preventDefault()}` on toolbar buttons keeps focus in the textarea.
- Trim spaces at the edges of the selection: double-click often selects "word ", and `**word **` does not render as bold.
- Italic uses `_` instead of `*`, so the italic toggle can't mistake half of a `**bold**` marker for its own.

```
const apply = (edit) => {
  const ta = ref.current;
  ta.focus();
  ta.setSelectionRange(edit.from, edit.to);
  if (!document.execCommand('insertText', false, edit.insert)) {
    ta.setRangeText(edit.insert, edit.from, edit.to);
    onChange(ta.value);
  }
  ta.setSelectionRange(edit.selStart, edit.selEnd);
};
```

`execCommand` is marked deprecated, but `insertText` is still the only way to edit a textarea while keeping native undo, and every major browser supports it. The alternative is our own undo stack (Q12).

Challenge 4: Parsing in the right order, without blocking typing

Problem. Inline rules interfere with each other. `` `**not bold**` `` must stay literal inside code. A URL like `https://x.io/a**b**` must not get `<strong>` injected into its `href`. `snake_case_name` must not become italic. And parsing the whole document on every keystroke can make typing feel laggy on a long document.

Solution.

- **Blocks first, then inline.** Walk the lines once: fences, headings, rules, quotes (recursive), lists, paragraphs. Code block bodies are escaped and never inline-parsed.
- **Park finished HTML behind placeholders.** Code spans and links are turned into HTML first and replaced with a `\u0000N\u0000` token, so the emphasis regexes can't see inside them. Then restore the tokens. Any `\u0000` in user input is replaced first so it can't fake a token.
- Underscore italic only at word boundaries; `*` italic must not start with a space (`2 * 3 * 4` stays as is).
- **`useDeferredValue` + `useMemo`.** The textarea updates with every key press at high priority. The preview receives a deferred copy, so React can skip intermediate renders when the user types fast. `memo(Preview)` and `useMemo` avoid re-parsing when only the footer changes.

```
export function inline(raw) {
  const tokens = [];
  const park = (html) => `\u0000${tokens.push(html) - 1}\u0000`;
  const restore = (s) => s.replace(/\u0000(\d+)\u0000/g, (_, n) => restore(tokens[n]));

  let s = escapeHtml(raw.replace(/\u0000/g, '\uFFFD'));
  s = s.replace(/`([^`]+)`/g, (_, code) => park(`<code>${code}</code>`));
  s = s.replace(/\[([^\]]+)\]\(([^)\s]+)\)/g, (_, text, url) =>
    park(`<a href="${safeUrl(url)}" target="_blank" rel="noopener noreferrer">${emphasis(text)}</a>`));
```

```
  return restore(emphasis(s));
}
```

---

8. Implement in the correct order

#	Step	Working result
1	`App` with `text` state, a textarea and a `<pre>{text}</pre>` preview	Typing shows up on the right
2	`markdownToHtml` with paragraphs and headings, rendered with `dangerouslySetInnerHTML`	Headings render
3	`escapeHtml` **before** anything else; test with `<img onerror>`	Raw HTML shows as text
4	`inline()`: code spans with placeholders, bold, italic, strike	Inline formatting works, code stays literal
5	Links + `safeUrl`	Links work; `javascript:` becomes `#`
6	Lists (bullets, numbers, tasks), quotes, fenced code, rules	All block syntax renders
7	`toggleWrap` + `apply` with `execCommand`; Bold button	Bold toggles, Ctrl+Z works, selection kept
8	`TOOLS` config + `Toolbar`; `toggleLinePrefix`, `insertLink`; shortcuts	Full toolbar
9	`useDraft`: load, debounced save, `pagehide`, error status	Refresh keeps the draft
10	`continueList` on Enter (skip during IME)	Lists continue and end naturally
11	View modes, word count, `useDeferredValue`, `memo`	Polish + performance
12	Accessibility pass: labels, `role="toolbar"`, `aria-pressed`, `role="status"`, focus rings	Keyboard and screen reader friendly

---

9. Cover all states and edge cases

State / edge case	What the user sees / what happens
First visit (nothing stored)	Sample document that shows every feature, including the XSS examples
Restored draft	Last saved text, status "Saved"
Empty document	Preview: "Nothing to preview yet"; 0 words; toolbar still works (inserts placeholders)
Typing	"Saving…" until 500 ms after the last key, then "Saved"
Storage full / blocked / private mode	Read falls back to the sample; status "Not saved…" in red; editing continues
Tab closed within 500 ms of typing	`pagehide` saves synchronously

State / edge case	What the user sees / what happens
`<script>`, `<img onerror>`, `<iframe>` in text	Shown as text
`javascript:`, `data:`, `JaVaScRiPt:`, `java\u0001script:` links	`href="#"`
`" onmouseover="` inside a link URL	Not a link (spaces not allowed), shown as text
Markdown inside inline code	Left as is
`**` inside a link URL	Not formatted (link is parked first)
`snake_case`, `2 * 3 * 4`	No accidental italics
`***both***`	Bold + italic
Unclosed code fence	Code block runs to the end (GitHub behavior)
Windows line endings `\r\n`	Normalized to `\n` before parsing
Ordered list starting at 3	`<ol start="3">`
Selection with leading / trailing spaces	Markers go around the word only
No selection + Bold	`**bold text**` with the placeholder selected
Bold pressed twice	Toggles off (markers inside or just outside the selection)
List prefix on a selection that ends at the start of a line	That last line is not prefixed
Enter in list / on empty item / during IME	Continue / exit / untouched
Ctrl+Z after toolbar action	Undone (native undo stack)
Very long document	Typing stays smooth (deferred preview); parser is linear in lines
Very long single line	Preview wraps (`overflow-wrap: anywhere`); code blocks scroll horizontally
Narrow screen in Split view	Editor and preview stack vertically

10. Final structure

```
_77_rich_text_markdown_editor/
├─ index.html
├─ package.json
├─ vite.config.js
└─ src/
   ├─ main.jsx
   ├─ index.css            # Tailwind + styles for the generated preview HTML
   ├─ App.jsx              # text, view mode, layout, footer
   ├─ hooks/
   │  └─ useDraft.js       # localStorage autosave (debounced + pagehide)
```

```
   ├─ utils/
   │  ├─ markdown.js        # escapeHtml, safeUrl, inline, markdownToHtml
   │  └─ editing.js         # toggleWrap, toggleLinePrefix, insertLink, continueList, TOOLS
   └─ components/
      ├─ Editor.jsx         # textarea, apply(edit), shortcuts, Enter in lists
      ├─ Toolbar.jsx
      └─ Preview.jsx
```

Setup: `npm create vite@latest` (React, JavaScript), then `npm i tailwindcss @tailwindcss/vite`.

---

11. Full working code

vite.config.js

```
import { defineConfig } from 'vite';
import react from '@vitejs/plugin-react-swc';
import tailwindcss from '@tailwindcss/vite';

export default defineConfig({
  plugins: [react(), tailwindcss()],
});
```

src/index.css

```
@import "tailwindcss";

/* Styles for the generated preview HTML (Tailwind classes can't be added to it). */
.markdown { line-height: 1.65; overflow-wrap: anywhere; }
.markdown > * + * { margin-top: 0.75rem; }
.markdown h1 { font-size: 1.75rem; font-weight: 700; }
.markdown h2 { font-size: 1.35rem; font-weight: 700; border-bottom: 1px solid #e2e8f0; padding-bottom:
0.25rem; }
.markdown h3 { font-size: 1.15rem; font-weight: 600; }
.markdown ul { list-style: disc; padding-left: 1.5rem; }
.markdown ol { list-style: decimal; padding-left: 1.5rem; }
.markdown li.task { list-style: none; margin-left: -1.25rem; }
.markdown a { color: #2563eb; text-decoration: underline; }
.markdown blockquote { border-left: 4px solid #cbd5e1; padding-left: 1rem; color: #475569; }
.markdown code { font-family: ui-monospace, monospace; font-size: 0.9em; background: #f1f5f9; padding:
0.1rem 0.3rem; border-radius: 0.25rem; }
.markdown pre { background: #0f172a; color: #e2e8f0; padding: 0.75rem 1rem; border-radius: 0.5rem;
overflow-x: auto; }
.markdown pre code { background: none; padding: 0; }
.markdown hr { border-color: #e2e8f0; }
```

src/main.jsx

```
import { StrictMode } from 'react';
import { createRoot } from 'react-dom/client';
import './index.css';
import App from './App';

createRoot(document.getElementById('root')).render(
  <StrictMode>
    <App />
  </StrictMode>
);
```

src/utils/markdown.js

```
// A small Markdown -> HTML converter for the preview.
// Safety rule: every piece of user text is escaped BEFORE any tag is added,
// so the only tags in the output are the ones this file writes.
// shortcut: regex-based subset (no nested lists, tables, images, HTML blocks); use marked + DOMPurify
for full CommonMark.

const ESCAPES = { '&': '&amp;', '<': '&lt;', '>': '&gt;', '"': '&quot;', "'": '&#39;' };
export const escapeHtml = (s) => s.replace(/[&<>"']/g, (c) => ESCAPES[c]);

// Allow http(s), mailto and relative links. Anything else (javascript:, data:, vbscript:) becomes "#".
export function safeUrl(url) {
  // Browsers ignore control chars and spaces inside a scheme ("java\u0001script:"), so strip them
before checking.
  const clean = url.replace(/[\u0000- \u007F]/g, '');
  const scheme = clean.match(/^([a-z][a-z0-9+.-]*):/i);
  if (!scheme) return url;
  return /^(https?|mailto)$/i.test(scheme[1]) ? url : '#';
}

const emphasis = (s) =>
  s
    .replace(/\*\*\*(.+?)\*\*\*/g, '<strong><em>$1</em></strong>')
    .replace(/\*\*(.+?)\*\*/g, '<strong>$1</strong>')
    .replace(/__(.+?)__/g, '<strong>$1</strong>')
    .replace(/~~(.+?)~~/g, '<del>$1</del>')
    .replace(/\*([^*\s][^*]*?)\*/g, '<em>$1</em>')
    .replace(/(^|[^\w])_([^_\s][^_]*?)_(?!\w)/g, '$1<em>$2</em>');

// Inline syntax for one line of text: `code`, [links](url), **bold**, _italic_, ~~strike~~.
export function inline(raw) {
  const tokens = [];
  // Finished HTML (code spans, links) is parked behind a placeholder so later regexes can't touch it.
```

```
  const park = (html) => `\u0000${tokens.push(html) - 1}\u0000`;
  const restore = (s) => s.replace(/\u0000(\d+)\u0000/g, (_, n) => restore(tokens[n]));

  let s = escapeHtml(raw.replace(/\u0000/g, '\uFFFD'));
  s = s.replace(/`([^`]+)`/g, (_, code) => park(`<code>${code}</code>`));
  s = s.replace(/\[([^\]]+)\]\(([^)\s]+)\)/g, (_, text, url) =>
    park(`<a href="${safeUrl(url)}" target="_blank" rel="noopener noreferrer">${emphasis(text)}</a>`)
  );
  return restore(emphasis(s));
}

const FENCE = /^```/;
const HEADING = /^(#{1,6})\s+(.*?)(?:\s+#+)?\s*$/;
const RULE = /^\s*([-*_])(\s*\1){2,}\s*$/;
const QUOTE = /^>\s?/;
const BULLET = /^\s*[-*+]\s+(.*)$/;
const NUMBER = /^\s*(\d+)[.)]\s+(.*)$/;
const TASK = /^\[([ xX])\]\s+(.*)$/;

const startsBlock = (line) =>
  FENCE.test(line) || HEADING.test(line) || RULE.test(line) || QUOTE.test(line) || BULLET.test(line) ||
NUMBER.test(line);

function listItem(text) {
  const task = text.match(TASK);
  if (!task) return `<li>${inline(text)}</li>`;
  const checked = task[1] !== ' ' ? ' checked' : '';
  return `<li class="task"><input type="checkbox" disabled${checked}> ${inline(task[2])}</li>`;
}

export function markdownToHtml(src) {
  const lines = src.replace(/\r\n?/g, '\n').split('\n');
  const out = [];
  let i = 0;

  // Collect consecutive lines that match `re` (and stop at the first that doesn't).
  const take = (re) => {
    const block = [];
    while (i < lines.length && re.test(lines[i])) block.push(lines[i++]);
    return block;
  };

  while (i < lines.length) {
    const line = lines[i];

    if (!line.trim()) {
      i++;
```

```
    } else if (FENCE.test(line)) {
      const lang = line.slice(3).trim().match(/^[\w-]*/)[0];
      const body = [];
      i++;
      while (i < lines.length && !FENCE.test(lines[i])) body.push(lines[i++]);
      i++; // closing fence (an unclosed fence runs to the end, like GitHub)
      const cls = lang ? ` class="language-${lang}"` : '';
      out.push(`<pre><code${cls}>${escapeHtml(body.join('\n'))}</code></pre>`);
    } else if (HEADING.test(line)) {
      const [, hashes, text] = line.match(HEADING);
      out.push(`<h${hashes.length}>${inline(text)}</h${hashes.length}>`);
      i++;
    } else if (RULE.test(line)) {
      out.push('<hr>');
      i++;
    } else if (QUOTE.test(line)) {
      const inner = take(QUOTE).map((l) => l.replace(QUOTE, ''));
      out.push(`<blockquote>${markdownToHtml(inner.join('\n'))}</blockquote>`);
    } else if (BULLET.test(line)) {
      const items = take(BULLET).map((l) => listItem(l.match(BULLET)[1]));
      out.push(`<ul>${items.join('')}</ul>`);
    } else if (NUMBER.test(line)) {
      const first = Number(line.match(NUMBER)[1]);
      const items = take(NUMBER).map((l) => listItem(l.match(NUMBER)[2]));
      out.push(`<ol${first !== 1 ? ` start="${first}"` : ''}>${items.join('')}</ol>`);
    } else {
      const para = [];
      while (i < lines.length && lines[i].trim() && !startsBlock(lines[i])) para.push(lines[i++]);
      out.push(`<p>${para.map(inline).join('<br>')}</p>`);
    }
  }
  return out.join('\n');
}
```

src/utils/editing.js

```
// Pure text edits for the toolbar. Each one takes (value, selectionStart, selectionEnd)
// and returns an Edit: replace value[from, to) with `insert`, then select [selStart, selEnd).
// Returning a range instead of the whole new string lets the editor apply it with
// execCommand('insertText'), which keeps the browser's undo history.

// Bold, italic, strike, inline code. Pressing it again on wrapped text removes the markers.
export function toggleWrap(value, start, end, marker, placeholder) {
  // Double-click often selects a trailing space; "**word **" would not render as bold.
  while (start < end && value[start] === ' ') start++;
  while (end > start && value[end - 1] === ' ') end--;
  const m = marker.length;
```

```
  const selected = value.slice(start, end);

  // Markers just outside the selection: **[word]**
  if (value.slice(start - m, start) === marker && value.slice(end, end + m) === marker) {
    return { from: start - m, to: end + m, insert: selected, selStart: start - m, selEnd: end - m };
  }
  // Markers inside the selection: [**word**]
  if (selected.length >= 2 * m && selected.startsWith(marker) && selected.endsWith(marker)) {
    const inner = selected.slice(m, -m);
    return { from: start, to: end, insert: inner, selStart: start, selEnd: start + inner.length };
  }
  const inner = selected || placeholder;
  return { from: start, to: end, insert: marker + inner + marker, selStart: start + m, selEnd: start +
m + inner.length };
}

// Heading, quote and lists: add the prefix to every line the selection touches,
// or remove it if all of those lines already have it.
export function toggleLinePrefix(value, start, end, prefix) {
  const prefixFor = typeof prefix === 'function' ? prefix : () => prefix;
  const from = value.lastIndexOf('\n', start - 1) + 1;
  // A selection that ends right after a newline doesn't include the next line.
  const last = end > start && value[end - 1] === '\n' ? end - 1 : end;
  const nl = value.indexOf('\n', last);
  const to = nl === -1 ? value.length : nl;

  const lines = value.slice(from, to).split('\n');
  const all = lines.every((line, i) => line.startsWith(prefixFor(i)));
  const insert = lines.map((line, i) => (all ? line.slice(prefixFor(i).length) : prefixFor(i) +
line)).join('\n');

  // Caret only: put it at the end of the line so typing continues there.
  if (start === end) return { from, to, insert, selStart: from + insert.length, selEnd: from +
insert.length };
  return { from, to, insert, selStart: from, selEnd: from + insert.length };
}

// [selected text](https://) with "https://" selected, so the user types the URL straight away.
export function insertLink(value, start, end) {
  const text = value.slice(start, end) || 'link text';
  const insert = `[${text}](https://)`;
  const urlStart = start + text.length + 3;
  return { from: start, to: end, insert, selStart: urlStart, selEnd: urlStart + 'https://'.length };
}

const LIST_LINE = /^(\s*)([-*+]|\d+[.])\s+(\[[ xX]\]\s+)?/;
```

```
// Enter inside a list item starts the next item; Enter on an empty item leaves the list.
// Returns null when Enter should behave normally.
export function continueList(value, start, end) {
  if (start !== end) return null;
  const lineStart = value.lastIndexOf('\n', start - 1) + 1;
  const nl = value.indexOf('\n', start);
  const before = value.slice(lineStart, start);
  const after = value.slice(start, nl === -1 ? value.length : nl);
  const match = before.match(LIST_LINE);
  if (!match) return null;

  if (match[0] === before && !after.trim()) {
    return { from: lineStart, to: start, insert: '', selStart: lineStart, selEnd: lineStart };
  }
  const [, indent, marker, task] = match;
  const next = /\d/.test(marker) ? `${parseInt(marker, 10) + 1}${marker.slice(-1)}` : marker;
  const insert = `\n${indent}${next} ${task ? '[ ] ' : ''}`;
  return { from: start, to: start, insert, selStart: start + insert.length, selEnd: start +
insert.length };
}

export const TOOLS = [
  { id: 'bold', label: 'Bold', icon: 'B', key: 'b', run: (v, s, e) => toggleWrap(v, s, e, '**', 'bold
text') },
  { id: 'italic', label: 'Italic', icon: 'I', key: 'i', run: (v, s, e) => toggleWrap(v, s, e, '_',
'italic text') },
  { id: 'strike', label: 'Strikethrough', icon: 'S', run: (v, s, e) => toggleWrap(v, s, e, '~~',
'text') },
  { id: 'code', label: 'Inline code', icon: '</>', run: (v, s, e) => toggleWrap(v, s, e, '`', 'code')
},
  { id: 'link', label: 'Link', icon: 'Link', key: 'k', run: insertLink },
  { id: 'heading', label: 'Heading', icon: 'H', run: (v, s, e) => toggleLinePrefix(v, s, e, '## ') },
  { id: 'quote', label: 'Quote', icon: '❝', run: (v, s, e) => toggleLinePrefix(v, s, e, '> ') },
  { id: 'ul', label: 'Bulleted list', icon: '•', run: (v, s, e) => toggleLinePrefix(v, s, e, '- ') },
  { id: 'ol', label: 'Numbered list', icon: '1.', run: (v, s, e) => toggleLinePrefix(v, s, e, (i) =>
`${i + 1}. `) },
  { id: 'task', label: 'Task list', icon: '☐', run: (v, s, e) => toggleLinePrefix(v, s, e, '- [ ] ') },
];
```

src/hooks/useDraft.js

```
import { useEffect, useState } from 'react';

function read(key, fallback) {
  try {
    return localStorage.getItem(key) ?? fallback;
  } catch {
```

```
    return fallback; // storage blocked (private mode, sandboxed iframe)
  }
}

// Text that autosaves to localStorage 500 ms after the user stops typing,
// and right away when the page is being closed so the last keystrokes aren't lost.
export function useDraft(key, fallback) {
  const [text, setText] = useState(() => read(key, fallback));
  const [status, setStatus] = useState('saved'); // 'saved' | 'saving' | 'error'

  useEffect(() => {
    const save = () => {
      try {
        localStorage.setItem(key, text);
        return true;
      } catch {
        return false; // quota exceeded or storage blocked
      }
    };
    const id = setTimeout(() => setStatus(save() ? 'saved' : 'error'), 500);
    window.addEventListener('pagehide', save);
    return () => {
      clearTimeout(id);
      window.removeEventListener('pagehide', save);
    };
  }, [key, text]);

  const update = (next) => {
    setText(next);
    setStatus('saving');
  };
  return [text, update, status];
}
```

src/components/Toolbar.jsx

```
import { TOOLS } from '../utils/editing';

const isMac = typeof navigator !== 'undefined' && /Mac|iPhone|iPad/.test(navigator.platform);

export default function Toolbar({ onTool }) {
  return (
    <div role="toolbar" aria-label="Formatting" className="flex flex-wrap gap-1 border-b border-slate-
200 bg-slate-50 p-1.5">
      {TOOLS.map((tool) => {
        const shortcut = tool.key ? ` (${isMac ? '⌘' : 'Ctrl+'}${tool.key.toUpperCase()})` : '';
        return (
```

```
          <button
            key={tool.id}
            type="button"
            aria-label={tool.label}
            title={tool.label + shortcut}
            // Keep focus (and the selection) in the textarea while clicking.
            onMouseDown={(e) => e.preventDefault()}
            onClick={() => onTool(tool)}
            className={`min-w-8 rounded px-2 py-1 text-sm text-slate-700 hover:bg-slate-200 focus-
visible:outline-2 focus-visible:outline-blue-500 ${
              tool.id === 'bold' ? 'font-bold' : tool.id === 'italic' ? 'italic' : tool.id === 'strike'
? 'line-through' : ''
            }`}
          >
            {tool.icon}
          </button>
        );
      })}
    </div>
  );
}
```

src/components/Editor.jsx

```
import { useRef } from 'react';
import Toolbar from './Toolbar';
import { TOOLS, continueList } from '../utils/editing';

export default function Editor({ value, onChange }) {
  const ref = useRef(null);

  // Apply an Edit ({ from, to, insert, selStart, selEnd }) to the textarea.
  const apply = (edit) => {
    const ta = ref.current;
    ta.focus();
    ta.setSelectionRange(edit.from, edit.to);
    // insertText goes through the browser's undo stack, so Ctrl+Z undoes a toolbar action.
    // It fires a normal input event, which reaches onChange below.
    // setRangeText is the fallback (no undo) for browsers where execCommand fails.
    if (!document.execCommand('insertText', false, edit.insert)) {
      ta.setRangeText(edit.insert, edit.from, edit.to);
      onChange(ta.value);
    }
    ta.setSelectionRange(edit.selStart, edit.selEnd);
  };

  const runTool = (tool) => {
```

```
    const ta = ref.current;
    apply(tool.run(ta.value, ta.selectionStart, ta.selectionEnd));
  };

  const onKeyDown = (e) => {
    const ta = e.currentTarget;
    if ((e.ctrlKey || e.metaKey) && !e.altKey && !e.shiftKey) {
      const tool = TOOLS.find((t) => t.key === e.key.toLowerCase());
      if (tool) {
        e.preventDefault();
        runTool(tool);
      }
      return;
    }
    // Don't touch Enter while an IME (Chinese, Japanese...) is composing a word.
    if (e.key === 'Enter' && !e.shiftKey && !e.nativeEvent.isComposing) {
      const edit = continueList(ta.value, ta.selectionStart, ta.selectionEnd);
      if (edit) {
        e.preventDefault();
        apply(edit);
      }
    }
  };

  return (
    <div className="flex min-h-0 flex-1 flex-col">
      <Toolbar onTool={runTool} />
      <label htmlFor="md-source" className="sr-only">
        Markdown source
      </label>
      <textarea
        id="md-source"
        ref={ref}
        value={value}
        onChange={(e) => onChange(e.target.value)}
        onKeyDown={onKeyDown}
        spellCheck
        placeholder="Write Markdown here…"
        className="min-h-64 flex-1 resize-none p-4 font-mono text-sm leading-6 text-slate-800 outline-
none"
      />
    </div>
  );
}
```

src/components/Preview.jsx

```
import { memo, useMemo } from 'react';
import { markdownToHtml } from '../utils/markdown';

// memo + useMemo: only re-parse when the (deferred) source text changes.
function Preview({ source }) {
  const html = useMemo(() => markdownToHtml(source), [source]);

  if (!source.trim()) {
    return <p className="p-4 text-sm text-slate-400">Nothing to preview yet.</p>;
  }
  return (
    <div
      className="markdown p-4"
      // Safe only because markdownToHtml escapes all user text before adding tags.
      dangerouslySetInnerHTML={{ __html: html }}
    />
  );
}

export default memo(Preview);
```

src/App.jsx

```
import { useDeferredValue, useState } from 'react';
import Editor from './components/Editor';
import Preview from './components/Preview';
import { useDraft } from './hooks/useDraft';

const SAMPLE = `# Release notes

Write **Markdown** on the left, see it on the right. Try _italic_, ~~strike~~ and \`inline code\`.

## Checklist
- [x] Toolbar with shortcuts (Ctrl+B, Ctrl+I, Ctrl+K)
- [ ] Press Enter in a list to continue it

1. Select text
2. Click a toolbar button

> Quotes work too.

\`\`\`js
const greet = (name) => \`Hi \${name}\`;
\`\`\`
```

```
These are shown as text, not run: <img src=x onerror="alert(1)"> and [a bad link](javascript:alert(1)).

Read the [Markdown guide](https://www.markdownguide.org).
`;

const VIEWS = [
  { id: 'write', label: 'Write' },
  { id: 'split', label: 'Split' },
  { id: 'preview', label: 'Preview' },
];

const SAVE_TEXT = { saved: 'Saved', saving: 'Saving…', error: 'Not saved: storage is full or blocked'
};

export default function App() {
  const [text, setText, saveStatus] = useDraft('md-editor:draft', SAMPLE);
  const [view, setView] = useState('split');
  // Typing stays instant; the preview re-renders with a slightly older value when React is busy.
  const deferredText = useDeferredValue(text);

  const words = text.trim() ? text.trim().split(/\s+/).length : 0;

  return (
    <div className="flex h-screen flex-col bg-slate-100 p-3 text-slate-900 sm:p-6">
      <header className="mb-3 flex flex-wrap items-center justify-between gap-2">
        <h1 className="text-lg font-semibold">Markdown editor</h1>
        <div role="group" aria-label="View" className="flex rounded-lg border border-slate-300 bg-white
p-0.5">
          {VIEWS.map((v) => (
            <button
              key={v.id}
              type="button"
              aria-pressed={view === v.id}
              onClick={() => setView(v.id)}
              className={`rounded-md px-3 py-1 text-sm ${view === v.id ? 'bg-slate-800 text-white' :
'text-slate-600 hover:bg-slate-100'}`}
            >
              {v.label}
            </button>
          ))}
        </div>
      </header>

      <main className={`grid min-h-0 flex-1 gap-3 ${view === 'split' ? 'grid-rows-2 md:grid-cols-2
md:grid-rows-1' : ''}`}>
        {view !== 'preview' && (
          <section aria-label="Editor" className="flex min-h-0 flex-col overflow-hidden rounded-lg
```

```
border border-slate-300 bg-white">
            <Editor value={text} onChange={setText} />
          </section>
        )}
        {view !== 'write' && (
          <section aria-label="Preview" className="min-h-0 overflow-auto rounded-lg border border-
slate-300 bg-white">
            <Preview source={view === 'preview' ? text : deferredText} />
          </section>
        )}
      </main>

      <footer className="mt-2 flex justify-between text-xs text-slate-500">
        <span>
          {words} words · {text.length} characters
        </span>
        <span role="status" className={saveStatus === 'error' ? 'text-red-600' : ''}>
          {SAVE_TEXT[saveStatus]}
        </span>
      </footer>
    </div>
  );
}
```

---

12. Wrap-up

Edge cases handled

- XSS through raw HTML, event-handler attributes, `javascript:` / `data:` links, mixed-case and control-character schemes, attribute breakout, code-block language.
- Parser interference: code spans and links are protected from emphasis; snake_case and arithmetic don't turn italic; CRLF line endings; unclosed fences.
- Editing: trailing-space selections, empty selections, toggle on/off from inside or outside, multi-line prefixes, list continuation and exit, IME composition.
- Persistence: blocked or full storage, closing the tab before the debounce fires.

Trade-offs to mention

Decision	Why	Cost
Textarea + preview, not contentEditable	Browser handles caret, IME, undo, a11y; source is a clean string	Not WYSIWYG; formatting is visible as symbols
Hand-written regex parser	No dependency, easy to explain, safe by construction	Subset of Markdown; no nested lists, tables, images; some CommonMark corner cases differ

Decision	Why	Cost
Escape first instead of sanitize after	Simple rule: no user tag can ever appear	Can't allow any raw HTML (e.g. `<kbd>`) without a sanitizer
`execCommand('insertText')`	Keeps native undo for toolbar actions	API is marked deprecated (still supported everywhere)
`useDeferredValue` instead of debounce	No fixed delay; React adapts to device speed	On a very slow device the preview can lag visibly
`localStorage`	Zero setup	~5 MB limit, one device, no history, synchronous
Single line break → ` `	Matches what users expect in an editor	Differs from strict CommonMark

What to add with more time

- **Tests:** the parser and edit functions are pure, so a table of input → expected output in Vitest; React Testing Library for toolbar + undo; a fuzz test that random input never produces `<script` or `on\w+=`.
- Swap the parser for `marked` + `DOMPurify` (or `markdown-it`) to get full CommonMark, tables, images, nested lists.
- Syntax highlighting for code blocks (Shiki / Prism) loaded lazily.
- Scroll sync between editor and preview.
- Image paste / drop: upload, then insert `![alt](url)`.
- Server save with versions, conflict handling (ETag), and offline queue.
- Real-time collaboration with a CRDT (Yjs) and remote cursors.
- Very large documents: parse in a Web Worker, or re-parse only changed blocks.
- A WYSIWYG mode built on ProseMirror / Lexical that reads and writes the same Markdown.

---

13. Counter questions the interviewer may ask

REQUIREMENTS

Q1. Why not build it with contentEditable? Isn't that what "rich text" means?

For a Markdown editor the source is a string, and a textarea edits strings perfectly: caret, IME, spellcheck, undo, screen readers and plain-text paste all come free. contentEditable makes the DOM the source of truth, every browser writes different HTML, pasted HTML comes in unfiltered, and React re-renders move the caret. I'd use contentEditable only for true WYSIWYG, and then behind a document model (ProseMirror / Lexical), not raw DOM.

Q2. What if the requirement is WYSIWYG, like Notion?

Keep a JSON model of blocks (`{ id, type: 'paragraph' | 'heading' | 'list', children: [{ text, marks: ['bold'] }] }`), render it into a contentEditable root, listen to `beforeinput` (`insertText`, `deleteContentBackward`, `insertParagraph`, `formatBold`), `preventDefault`, and apply the change to the model. Map DOM selection to `{ blockId, offset }`. Undo is a stack of model operations. That is a multi-week project, so in an interview I'd name a library and explain these parts.

Q3. Why not just use a library like `marked`?

In real work I would, plus `DOMPurify` on its output. In the interview I wrote a small parser because the task is about showing I understand where XSS comes from. Also note: `marked` alone is **not** safe; it passes raw HTML through by default. The sanitizer is not optional.

STATE DESIGN

Q4. Why don't you store the HTML in state?

It is a pure function of the text. Storing it creates two sources of truth that can disagree, and an extra render per keystroke. `useMemo` gives the same caching without the drift.

Q5. Why isn't the selection in React state?

The textarea already owns it, and it changes on every click and arrow key. Mirroring it would cause constant re-renders and races with the browser. We read `selectionStart` / `selectionEnd` only when a tool runs, which is the only time we need them.

Q6. Controlled or uncontrolled textarea?

Controlled, because autosave, word count and preview all need the value in React. It is fine because the value we set always equals what the DOM already has (our edits go through the DOM first and come back through `onChange`), so React never resets the caret.

XSS AND THE PARSER

Q7. Is escaping enough? What about attributes?

Escaping `& < > " '` covers text and quoted attribute values. Two things need more: URL attributes (a fully escaped `javascript:alert(1)` still runs, hence the scheme allow-list) and anything in unquoted attributes, `style`, or inside `<script>`, which we never generate. That's why we only ever emit a fixed set of tags with quoted attributes.

Q8. Why an allow-list for URLs instead of blocking `javascript:`?

Block-lists always miss something: `JAVASCRIPT:`, `java\tscript:`, `data:text/html`, `vbscript:`, future schemes. An allow-list of `http`, `https`, `mailto` and relative is short and fails safe.

Q9. What if we want to allow some HTML, like `<kbd>` or `<details>`?

Then "escape everything" no longer works. Parse with HTML allowed, then sanitize the result with `DOMPurify` configured with an allow-list of tags and attributes (`ALLOWED_TAGS: ['kbd', 'details', 'summary', ...]`). Add a CSP (`script-src 'self'`) so a sanitizer bug can't run inline scripts.

Q10. Your preview is shown only to the author. Does XSS even matter?

Yes. The author may paste text from somewhere else (self-XSS through copy-paste is a real phishing trick), and once the doc is shared, saved to a server, or shown in a comment thread, it becomes stored XSS for every reader. Build it safe from day one; the server must also sanitize when it renders, since the client can be bypassed.

Q11. Why the `\u0000` placeholders in `inline()`?

So later regexes can't touch HTML we already built. Without it, `` `**x**` `` would get bold inside code, and a `**` inside a link URL would put `<strong>` into the `href`. We replace any `\u0000` in the input first, so the user can't forge a placeholder.

TOOLBAR AND EDITING

Q12. `execCommand` is deprecated. What's the alternative?

Keep our own undo stack: on each change push `{ value, selectionStart, selectionEnd }` (grouping keystrokes typed within ~1 s into one entry), intercept Ctrl+Z / Ctrl+Shift+Z, and restore. It works everywhere but you must also handle IME, paste and the Edit menu. Today `insertText` is still supported in all major browsers and is the cheaper choice; I'd wrap it so it can be swapped.

Q13. Why does clicking a toolbar button not lose the selection?

`onMouseDown={e => e.preventDefault()}` stops the button from taking focus, so the textarea keeps focus and its selection. Even if it did blur, a textarea keeps `selectionStart/End`, and `apply` calls `focus()` before editing. Keyboard users who Tab to the toolbar still work for the same reason.

Q14. How would you show which formats are active at the cursor (Bold button pressed)?

On `select` / `keyup`, look around the caret: is it between a `**` before and a `**` after on the same line? Set `aria-pressed` on the button. Throttle it with `requestAnimationFrame`. In a Markdown editor it's less important than in WYSIWYG, so I left it out.

Q15. Why `_` for italic and not `*`?

With `*`, pressing Italic on text inside `**bold**` would see one `*` on each side and remove half of the bold markers. `_` never collides with `**`. GitHub's toolbar makes the same choice.

Q16. What happens with Enter while typing in Chinese or Japanese?

During IME composition, Enter confirms the candidate word. If we ran list logic then, we'd break the input. We check `e.nativeEvent.isComposing` and leave Enter alone while composing.

PERFORMANCE

Q17. What if the document is 1 MB (a whole book)?

The parser is linear, but a full re-parse + full `innerHTML` replacement per keystroke gets heavy. Steps: `useDeferredValue` (already there), then split the doc into blocks and cache HTML per block (only changed blocks re-parse), render blocks as separate keyed elements so React only touches changed ones, and move parsing to a Web Worker. For the editor side, a textarea handles a few MB fine; beyond that, CodeMirror virtualizes lines.

Q18. Why `useDeferredValue` instead of debouncing the preview by 300 ms?

Debounce always waits 300 ms, even on a fast laptop, and still blocks if parsing is slow. `useDeferredValue` renders the preview as soon as there is idle time and drops stale renders when new keys arrive, so it is

instant on fast devices and smooth on slow ones. Debounce is still right for the **save**, which is I/O.

Q19. Does replacing `innerHTML` on every change cause problems?

It throws away preview DOM state: text selected in the preview, a playing video, scroll inside a code block. For a preview pane that's acceptable. If it matters, render the parsed blocks as React elements (or use a DOM diff like `morphdom`) instead of one big HTML string.

PERSISTENCE AND TRADE-OFFS

Q20. Why save on `pagehide` and not `beforeunload`?

`pagehide` fires reliably on mobile and when the page goes into the back/forward cache; `beforeunload` doesn't fire on many mobile tab closes and blocks bfcache. `localStorage.setItem` is synchronous, so it finishes before the page is gone. For a server save, use `navigator.sendBeacon` or `fetch(..., { keepalive: true })`.

Q21. The same draft is open in two tabs. What happens?

Last write wins, and one tab silently overwrites the other. Listen to the `storage` event: when another tab saves, either load its text (if this tab has no unsaved changes) or show "This draft was changed in another tab". With a server, send a version number and reject stale saves with 409.

Q22. How would you save to a server instead?

Debounce ~1 s, `PUT /docs/:id { content, version }`, with an `AbortController` so only the latest request matters, and show "Saving / Saved / Offline". Keep `localStorage` as an offline copy and retry when `online` fires. Sanitize on the server too when the doc is rendered for others.

Q23. How would you add real-time collaboration?

Not by sending the whole text: concurrent edits would overwrite each other. Use a CRDT like Yjs (`Y.Text`) bound to the editor, sync over WebSocket, and show remote cursors with awareness. Undo must become per-user (Yjs `UndoManager`). The preview still derives from the shared text, so it stays the same.

Q24. How would you test this?

Unit tests for the pure parts: a table of Markdown → HTML cases, a list of XSS payloads asserting no `<script`, `on\w+=` or `javascript:` in the output, and `toggleWrap` / `toggleLinePrefix` / `continueList` cases by applying the returned Edit to a string. Component tests with Testing Library: select text, click Bold, check the value and the selection. One Playwright test for Ctrl+Z after a toolbar action, since jsdom doesn't implement `execCommand`.